

# IBlockchainAdapter 멀티체인 추상화 설계

Strategy Pattern · Factory Pattern · ChainAdapterRegistry

강의 55min + 실습 30min

EVM → XRPL 교체

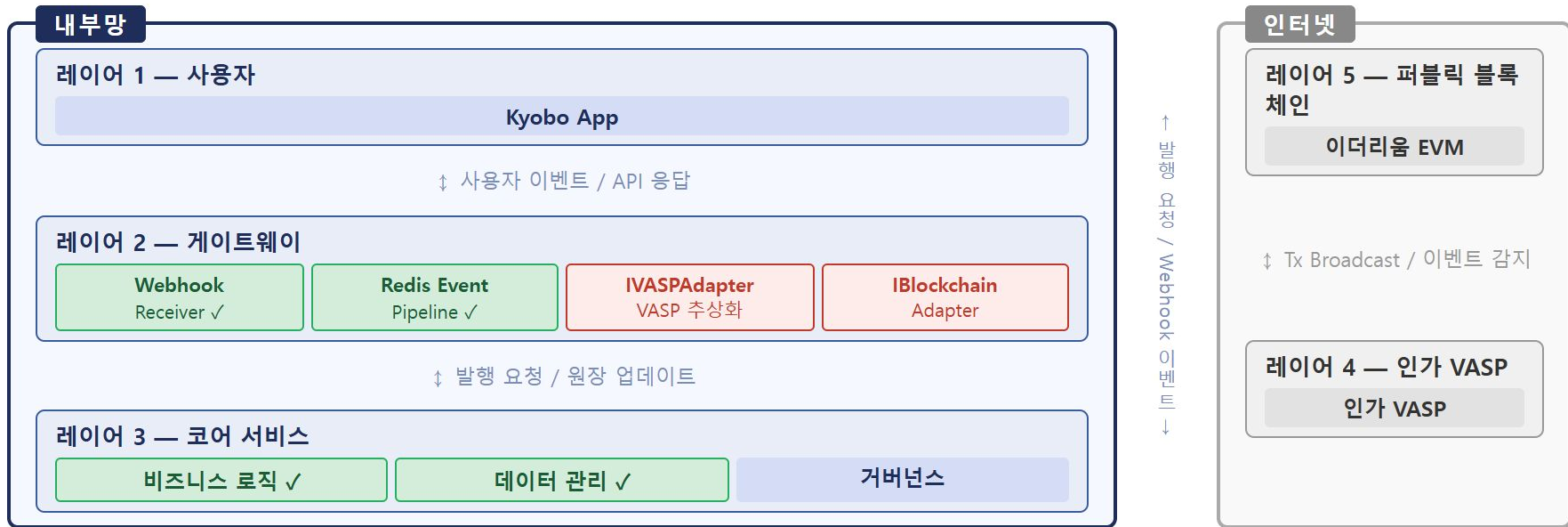
DI 한 줄

Phase 1 → 3 로드맵

**COINCRAFT**  
ACADEMY

# 전체 아키텍처 맵

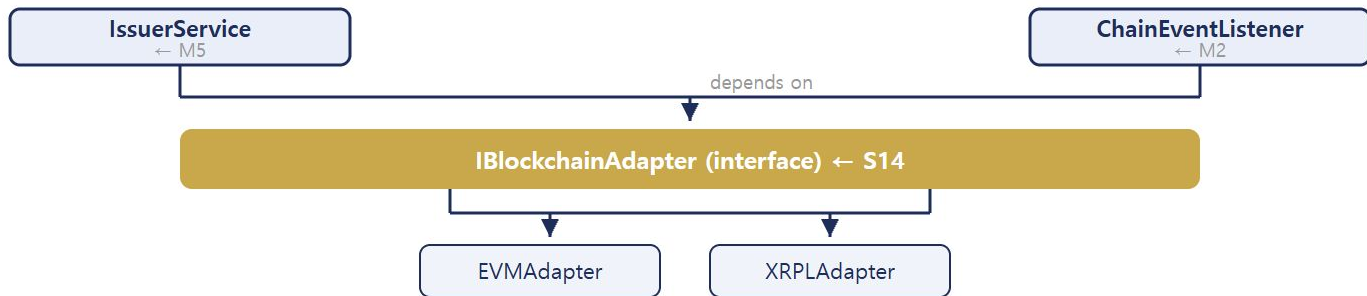
ARCHITECTURE REMINDER — 신뢰 경계로 시스템을 나눈다 · S14는 레이어 2 어댑터 추상화에 집중



S14 핵심 구간 (빨간 박스): IVASPAAdapter — VASP마다 다른 API를 단일 인터페이스로 감싼다. KyoboVASP·ExternalVASP 교체 시 상위 코드 무변경. | IBlockchainAdapter — 체인(EVM·XRPL·Solana)이 바뀌어도 TxStateMachineService는 동일 코드로 동작한다.

# S14 — IBlockchainAdapter 의존 구조

WHO DEPENDS ON IBlockchainAdapter — M2 · M3 · M5



| 인터페이스               | 설명                                                           | 교체 지점                  | 세션      |
|---------------------|--------------------------------------------------------------|------------------------|---------|
| IBlockchainAdapter  | 체인 연산 추상화 — sendTransaction-getReceipt-subscribeEvents       | EVM→XRPL→Circle        | S14     |
| IVASPAAdapter       | VASP API 추상화 — submitTransaction-getTransactionStatus        | KyoboVASP→ExternalVASP | S21     |
| VaspTxClient        | TX 실행 위임 — submitMint-getStatus-resubmitWithGasBump          | External→Custody       | S13     |
| ISignerService      | 서명 추상화 — sign(SignRequest). Phase 3 내재화 핵심 교체 지점             | 소프트웨어→HSM·KMS          | Phase 3 |
| ICoreBankingAdapter | 코어뱅크 추상화 — getUserAccount-syncBalance-sendRewardNotification | 내부 API 변경              | —       |
| IKYCProvider        | KYC 추상화 — getKYCStatus(userId) → KYCStatus                   | KYC 벤더 교체              | —       |

# EVM 고착화 리스크 1/2

## VENDOR LOCK-IN — WHAT IT MEANS

**고착화(Vendor Lock-in)** — 특정 기술·벤더의 개념이 비즈니스 코드 깊숙이 침투해, 나중에 그 기술을 바꾸려면 비즈니스 코드까지 전부 뜯어고쳐야 하는 상황.

### ❌ 전용 충전 케이블 (고착화)

충전 케이블을 더 빠른 걸로 바꾸고 싶은데  
노트북 단자가 전용 규격 → 노트북도 교체  
가방·거치대까지 연쇄 교체

→ 케이블 하나 바꾸려고 노트북까지 바꿈

### ✅ USB-C 표준 (추상화)

케이블을 바꿔도 노트북은 그대로  
노트북을 바꿔도 케이블은 그대로

→ 케이블 교체 비용 = 케이블 하나

**소프트웨어도 같다** — `IBlockchainAdapter` = 노트북의 USB-C 포트 규격 / `EVMAdapter·XRPLAdapter` = USB-C 케이블 / `IssuerService` = 노트북. 케이블(체인)을 바꿔도 포트 규격이 같으면 노트북(비즈니스 코드)은 그대로.

# EVM 고착화 리스크 2/2

## EVM-LOCKED CODE — REPLACEMENT COST

### ✗ EVM 고착 코드

```
const tx      = await ethers.provider
               .sendTransaction({...});

const receipt = await tx.wait();      // ethers.js 전용
const gasUsed = receipt.gasUsed;      // EVM 개념
const log     = receipt.logs[0];      // EVM Log
const topic   = log.topics[0];        // EVM event topic
const decoded = iface.parseLog(log);  // ABI 디코딩
```

### 이 코드를 XRPL로 교체하면...

- `xrpl.js` 사용법 전혀 다름
- `sendTransaction` 개념 없음 → XRPL 전용 트랜잭션 방식 사용
- Log / Topic 없음 → Transaction Metadata 파싱
- ABI 없음 → 별도 파싱 로직 필요
- → **IssuerService + LedgerService + AuditLog 전부 수정**

**결론** — 체인 전용 개념이 비즈니스 로직에 스며들면 교체 비용이 폭발한다. 추상화 레이어가 필요한 이유다.

# IBlockchainAdapter 설계 원칙 1/3

## STRATEGY PATTERN — DEFINITION & ANALOGY

**Strategy Pattern** — 같은 목적을 수행하는 여러 구현체를 인터페이스 뒤에 감추고, 런타임에 교체 가능하게 만드는 패턴.

### 비유 — 결제 수단

- 쇼핑몰 결제 시스템은 "결제"라는 행위만 안다
- 카드·카카오페이·무통장 — 결제 시스템 코드 변경 없음
- 결제 수단(구현체)만 바뀜



### 핵심 원칙

- 호출하는 쪽은 **인터페이스만** 안다
- 구현체(어댑터)는 런타임에 주입된다
- 구현체 교체 시 호출하는 쪽 코드 변경 없음
- 새 구현체 추가 = 새 클래스 하나 추가

**GoF 원칙** — "프로그램에서 변하는 부분을 찾아 나머지와 분리하라"

# IBlockchainAdapter 설계 원칙 2/3

## STRATEGY PATTERN — BLOCKCHAIN ADAPTER

블록체인 어댑터도 동일하다 — 비즈니스 레이어는 IBlockchainAdapter 인터페이스만 안다. EVM인지 XRPL인지 Circle인지 모른다.



교체 원칙 — 어댑터 교체 = DI 설정 한 줄 변경. 비즈니스 코드는 변경 없음.



# IBlockchainAdapter 설계 원칙 3/3

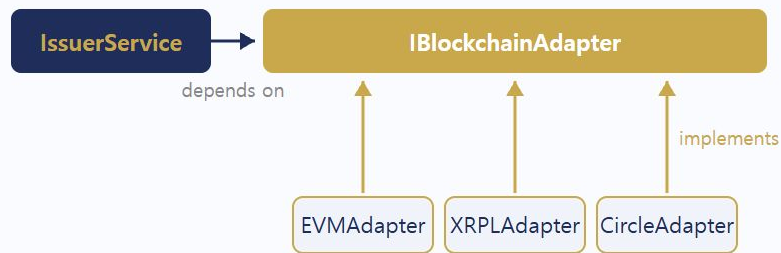
STRATEGY PATTERN — DI & DEPENDENCY GRAPH

## 실제 주입 코드 — TokenIssuerFactory.ts

```
// TokenIssuerFactory.createNFTIssuer()
const issuerService = new IssuerService({
  chainAdapter: this.deps.chainAdapter, // IBlockchainAdapter
  vaspAdapter: this.deps.vaspAdapter, // IVASPAAdapter
  coreBanking: this.deps.coreBanking, // ICoreBankingAdapter
  policyService,
  conditionService,
  issuanceRepo,
  txStateMachine,
  ledgerService,
});
```

```
// 체인 교체 = 주입하는 어댑터 구현체만 바꾸면 됨
// IssuerService · TokenIssuerFactory 코드는 한 줄도 변하지 않는다
```

## IssuerService 의존성 순서도



**핵심** — IssuerService는 인터페이스에만 의존. 어떤 어댑터가 주입되는지 관심 없다.

**Decorator 결합** — `RetryAdapterDecorator` · `LoggingAdapterDecorator` 도 동일 인터페이스를 구현. 비즈니스 코드 변경 없이 재시도·로깅 추가 가능.



# 실제 운영 코드의 주입 경로 1/6

DI IN PRODUCTION — CONCRETE ADAPTER CREATION

## ① 구체 구현체 생성 — index.ts:67

```
const chainAdapter = new EVMAdapter({
  rpcUrl:    process.env.RPC_URL!,
  chainId:   process.env.CHAIN_ID!,
  privateKey: process.env.OPERATOR_PRIVATE_KEY!,
});
```

### 핵심 포인트

`EVMAdapter` 가 코드베이스에서 **여기에만** 등장한다.

이 한 줄이 Phase 3에서

`XRPLAdapter` 또는 `ChainAdapterFactory.createFromEnv()` 로 교체되는 지점.

### 이후 코드에서 `EVMAdapter`가 보이지 않는 이유

- Factory 생성자 타입: `IBlockchainAdapter`
- IssuerService 생성자 타입: `IBlockchainAdapter`
- ChainEventListener 생성자 타입: `IBlockchainAdapter`
- 구체 클래스는 진입점(index.ts)만 알고 있음

# 실제 운영 코드의 주입 경로 2/6

## DI IN PRODUCTION — FACTORY CONSTRUCTOR INJECTION

### ② Factory 생성자에 주입 — index.ts:120

```
const factory = new TokenIssuerFactory({ chainAdapter, vaspAdapter, coreBanking, pool: pgPool });
```

### TokenIssuerFactory 생성자 타입 — TokenIssuerFactory.ts:44

```
constructor(  
  private readonly deps: {  
    chainAdapter: IBlockchainAdapter; // ← 타입은 인터페이스  
    vaspAdapter: IVASPAdapter;  
    coreBanking: ICoreBankingAdapter;  
    pool: Pool;  
  },  
) {}
```

**포인트** — EVMAAdapter 인스턴스가 IBlockchainAdapter 타입으로 받아진다. Factory 내부는 구체 클래스 이름을 모른다.

# 실제 운영 코드의 주입 경로 3/6

## DI IN PRODUCTION — ISSUERSERVICE INJECTION

### ③ IssuerService까지 전달 — TokenIssuerFactory.ts:84

```
const issuerService = new IssuerService({
  chainAdapter: this.deps.chainAdapter, // ← IBlockchainAdapter 그대로 전달
  vaspAdapter: this.deps.vaspAdapter,
  coreBanking: this.deps.coreBanking,
  // ...
});
```

### IssuerService 생성자 — IssuerService.ts:31

```
constructor(private readonly deps: {
  chainAdapter: IBlockchainAdapter; // ← 인터페이스 타입으로 수신
  // ...
}) {}
```

추상화 유지 — IssuerService는 `mintNFT()` · `getReceipt()` 등을 `IBlockchainAdapter` 인터페이스를 통해서만 호출한다. `ethers.js` import 없음.

# 실제 운영 코드의 주입 경로 4/6

DI IN PRODUCTION — CHAINEVENTLISTENER

## ④ ChainEventListener에도 직접 주입 — index.ts:138

```
const eventListener = new ChainEventListener(  
  chainAdapter, // ← IBlockchainAdapter  
  [nftIssuedHandler, issuanceConfirmHandler],  
);
```

### IBlockchainAdapter 메서드 — adapter를 통해 호출

- `adapter.subscribeEvents()` — 실시간 이벤트 수신 (WebSocket)
- `adapter.queryEvents()` — 재시작 후 누락 이벤트 복구 (Pull)
- ChainEventListener 자신의 메서드가 아님
- 주입받은 어댑터 인스턴스에 위임

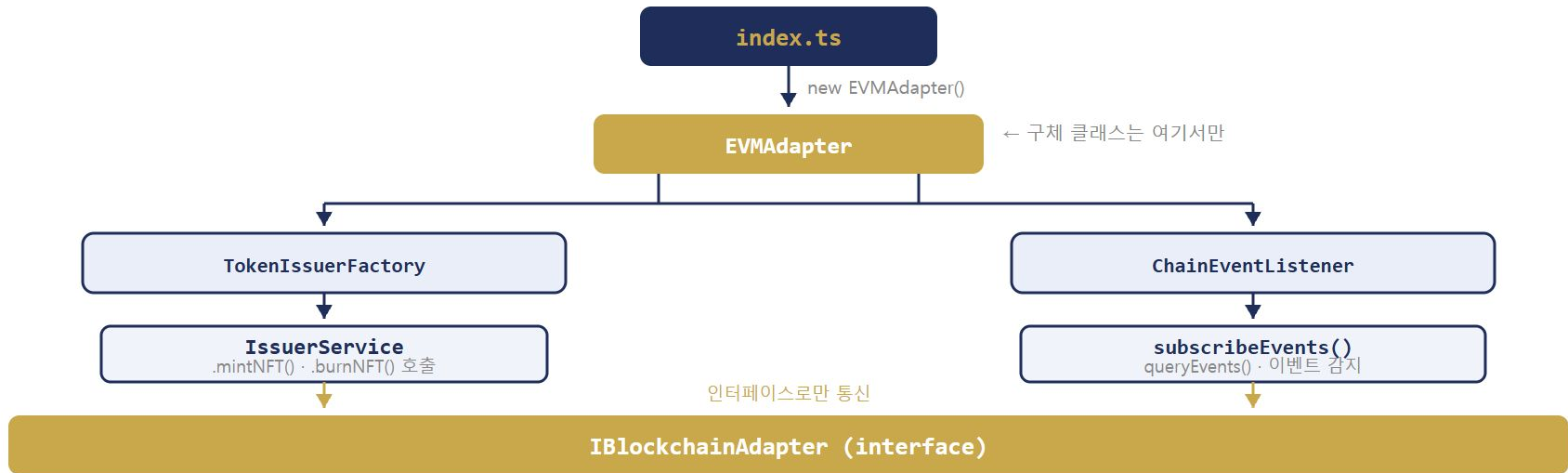
### 두 클래스의 역할

- **IssuerService** — NFT 발행 처리 (write)
- **ChainEventListener** — 온체인 이벤트 구독 (read)
- 둘 다 같은 어댑터 인스턴스를 공유
- 어댑터가 교체되어도 두 클래스 수정 없음

# 실제 운영 코드의 주입 경로 5/6

DI IN PRODUCTION — FULL FLOW SUMMARY

전체 흐름 정리



원칙 — 구체 클래스( `EVMAAdapter` )는 오직 `index.ts` 한 곳에서만 등장한다. 나머지 모든 코드는 `IBlockchainAdapter` 만 안다.

# 실제 운영 코드의 주입 경로 6/6

## DI IN PRODUCTION — PHASE 3 MIGRATION

### Phase 3에서 XRPL로 전환 시 변경점

#### ❌ Before (EVM)

```
// index.ts:67
const chainAdapter =
  new EVMAdapter({
    rpcUrl, chainId, privateKey
  });
```

#### ✅ After (XRPL)

```
// index.ts:67 — 이 한 줄만 교체
const chainAdapter =
  new XRPLAdapter({ wsUrl, seed });
// 또는
ChainAdapterFactory.createFromEnv('XRPL');
```

TokenIssuerFactory — 변경 없음 ✅

IssuerService — 변경 없음 ✅

ChainEventListener — 변경 없음 ✅

# Factory Pattern — "만드는 책임"의 분리 1/6

FACTORY PATTERN — CREATION RESPONSIBILITY SEPARATION

## Strategy Pattern vs Factory Pattern

- **Strategy** — "어떻게 사용할지"를 인터페이스 뒤로 감춤
- **Factory** — "어떻게 만들지"를 별도 클래스로 분리
- 둘은 함께 쓰이는 경우가 많음

## 비유 — 자동차 공장과 운전자

- **운전자(IssuerService)** — 운전법만 안다
- 차가 어떻게 조립됐는지 관심 없다
- **공장(TokenIssuerFactory)** — 조립만 책임진다
- 엔진(EVMAdapter) · 차대(VaspAdapter) · 계기판(PolicyService) 끼워 맞추

**핵심** — IssuerService 를 호출하는 쪽( index.ts )은 조립 과정을 전혀 모른다. factory.createNFTIssuer() 한 줄만 호출하면 된다.



# Factory Pattern — "만드는 책임"의 분리 2/6

FACTORY PATTERN — TWO FACTORIES, DIFFERENT ROLES

이 프로젝트에는 Factory가 두 개 있다. 역할이 다르다.

## ChainAdapterFactory

"어떤 어댑터를 만들지" 결정

- EVM → EVMAdapter 생성
- XRPL → XRPLAdapter 생성
- 위치: `chain-adapters/src/`

## TokenIssuerFactory

"서비스 전체를 어떻게 조립할지" 결정

- IssuerService 의존성 그래프 전체 조립
- PgRepo · PolicyService · TxStateMachine · LedgerService
- 위치: `issuer-service/src/factory/`

계층 관계 — ChainAdapterFactory가 만든 어댑터를 TokenIssuerFactory가 받아서 IssuerService 안에 주입한다.

# Factory Pattern — "만드는 책임"의 분리 3/6

## FACTORY PATTERN — WHAT IT GIVES US

| 역할      | 담당 클래스              | 알고 있는 것                           | 변경 범위    |
|---------|---------------------|-----------------------------------|----------|
| 어댑터 생성  | ChainAdapterFactory | EVM / XRPL / Circle / UTXO 구체 클래스 | 체인 추가 시  |
| 서비스 조립  | TokenIssuerFactory  | 의존성 그래프 전체                        | 의존성 변경 시 |
| 비즈니스 로직 | IssuerService       | IBlockchainAdapter 인터페이스만         | 변경 없음    |
| 이벤트 구독  | ChainEventListener  | IBlockchainAdapter 인터페이스만         | 변경 없음    |
| 진입점     | index.ts            | 어댑터 선택 + Factory 호출만              | 한 줄 교체   |

결과 — Phase 3 전환 시 `index.ts:67` 한 줄만 바꾸면 전체 체인이 교체된다. 세 역할이 분리되기 때문이다.

# Factory Pattern — "만드는 책임"의 분리 4/6

## FACTORY PATTERN — TOKENISSUERFACTORY INTERNALS

### TokenIssuerFactory.createNFTIssuer() 내부

```
createNFTIssuer() {  
  const policyRepo = new PgIssuancePolicyRepository(this.deps.pool);  
  const policyService = new IssuancePolicyService(policyRepo);  
  const txRepo = new PgTxRepository(this.deps.pool);  
  const vaspClient = new VaspTxClientAdapter(this.deps.vaspAdapter);  
  const stateMachine = new TxStateMachineService(txRepo, vaspClient);  
  const ledger = new LedgerService(this.deps.pool);  
  const bridge = new TxTransitionBridge(stateMachine, ledger);  
  
  return new IssuerService({  
    chainAdapter: this.deps.chainAdapter, // ← 인터페이스 타입 그대로  
    policyService, stateMachine, ledger, bridge, // ...  
  });  
}
```

포인트 — 이 복잡한 조립 과정을 `index.ts` 는 모른다. `factory.createNFTIssuer()` 한 줄이면 완성된 서비스를 받는다.

# Factory Pattern — "만드는 책임"의 분리 5/6

FACTORY PATTERN — CHAINADAPTERFACTORY INTERNALS

## ChainAdapterFactory.create()

```
static create(config: AdapterConfig): IBlockchainAdapter {
  switch (config.chainType) {
    case 'EVM': return new EVMAdapter({...});
    case 'XRPL': return new XRPLAdapter({...});
    default:
      const _: never = config.chainType; // 컴파일 타임 체크
      throw new UnsupportedChainError(_);
  }
}
```

## default: never 패턴이 중요한 이유

- ChainType 에 새 체인 추가 시
- 이 default 분기에서 컴파일 에러 발생
- switch 분기 추가 안 하면 빌드 실패
- 누락 방지를 타입 시스템이 강제

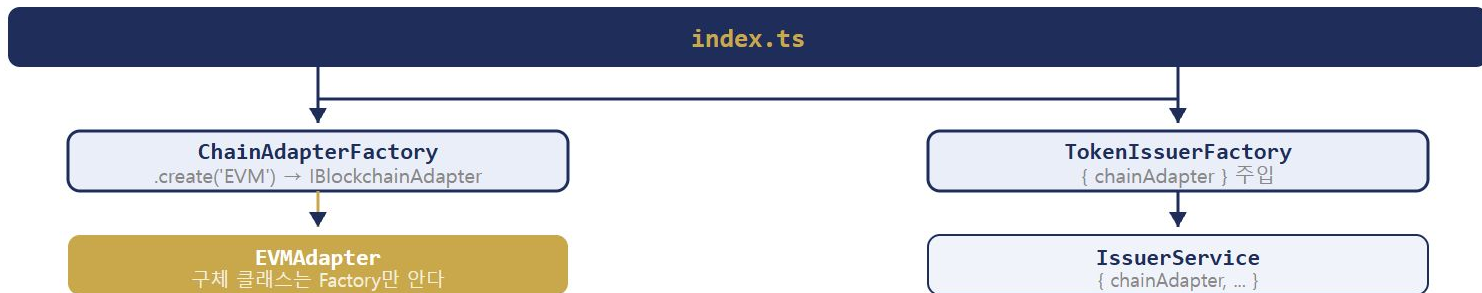
switch 분기가 ChainAdapterFactory에 갇혀 있다

index.ts 와 IssuerService 는 이 분기를 모른다.

# Factory Pattern — "만드는 책임"의 분리 6/6

## FACTORY PATTERN — TWO FACTORIES HIERARCHY

### 두 Factory의 계층 관계



| Factory             | 위치                          | 결정하는 것                          |
|---------------------|-----------------------------|---------------------------------|
| ChainAdapterFactory | chain-adapters/src/         | EVM / XRPL / Circle / UTXO 중 선택 |
| TokenIssuerFactory  | issuer-service/src/factory/ | IssuerService 의존성 그래프 조립        |

**Phase 3 전환** — `ChainAdapterFactory.createFromEnv('EVM')` → `createFromEnv('XRPL')` · `TokenIssuerFactory` · `IssuerService` · `ChainEventListener` 변경 없음.

# 파라미터 인터페이스 — 왜 별도로 분리하는가

## PARAMETER INTERFACES — SEPARATION RATIONALE

### 주요 파라미터 인터페이스

```
// IBlockchainAdapter.ts:50
interface MintParams {
  contractAddr: string; to: string;
  tokenId: bigint; amount: bigint;
  requestId: string; // Idempotency key
}
interface MintBatchParams {
  contractAddr: string; to: string[];
  tokenIds: bigint[]; amounts: bigint[];
}
interface ContractCallParams {
  contractAddr: string; abi: unknown[];
  method: string; args: unknown[];
  signerKey?: string; // 없으면 read-only
}
```

### 왜 인터페이스로 분리하는가

- Named parameters — 순서 실수 방지, 이름으로 전달
- 옵션 필드 추가 시 메서드 시그니처 변경 불필요
- 호출 측은 체인 종류 무관 — 동일한 구조로 전달
- 다음 장표 `IBlockchainAdapter` 메서드의 입력 타입

#### requestId가 MintParams에 있는 이유

S13의 UUID requestId가 여기까지 내려옴. 컨트랙트 calldata 포함 → 중복 차단 가능.



# IBlockchainAdapter 전체 인터페이스

## IBLOCKCHAINADAPTER — FULL INTERFACE OVERVIEW

### 메서드 그룹

```
// 연결 상태
isConnected(): Promise<boolean>
getBlockNumber(): Promise<number>

// NFT 발행·소각 (Phase 3 이후 활성화)
mintNFT(p: MintParams): Promise<TransactionReceipt>
mintNFTBatch(p: MintBatchParams): Promise<TransactionReceipt>
burnNFT(p: BurnParams): Promise<TransactionReceipt>
getBalance(addr, owner, id): Promise<bigint>

// 이벤트 (Phase 1 직접 사용)
subscribeEvents(...): Promise<() => void>
queryEvents(...): Promise<ChainEvent[]>
getReceipt(txHash): Promise<TransactionReceipt | null>
```

### 주목할 설계 포인트

- `subscribeEvents` → `unsubscribe` 함수 반환

GracefulShutdown 시 `unsub()` 한 줄

- `getReceipt` → `null` 가능

PENDING TX = null → TxStateMachine이 TIMEOUT 처리

- write 메서드 (`mintNFT` 등) → Phase 3까지 VASP가 대신 실행
- read 메서드 (`queryEvents` 등) → Phase 1부터 직접 사용

파라미터 — 모두 앞 장표의 인터페이스(`MintParams` 등)가 입력  
됨



# 공통 데이터 추상화 1/2

## COMMON DATA ABSTRACTION — TRANSACTIONRECEIPT

### TransactionReceipt — IBlockchainAdapter.ts:23

```
interface TransactionReceipt {  
  txHash:      string;  
  blockNumber: number;  
  blockHash:   string;  
  status:      'success' | 'failed' | 'pending';  
  gasUsed?:    bigint;    // EVM만 유효, XRPL은 undefined  
  timestamp:   number;  
}
```

**ethers.js와 혼동 주의** — ethers.js의 `TransactionReceipt` 와 다름. ethers.js에서 `gasUsed`는 필수 필드. 여기서 optional인 것은 XRPL 수용을 위한 설계 결정.

### gasUsed가 optional인 이유

- EVM: 가스 사용량 필수 → `gasUsed: bigint`
- XRPL: 가스 개념 없음 → `undefined`
- Circle: 가스 추상화 → `undefined`

**설계 원칙** — 어댑터가 체인 특성에 맞는 필드만 채운다. 상위 레이어는 `gasUsed`에 의존하지 않아야 한다.

**S13 연결** — `TxStateMachineService`가 receipt의 `null` 여부로 PENDING 판단.

# 공통 데이터 추상화 2/2

## COMMON DATA ABSTRACTION — CHAINEVENT & RAW FIELD

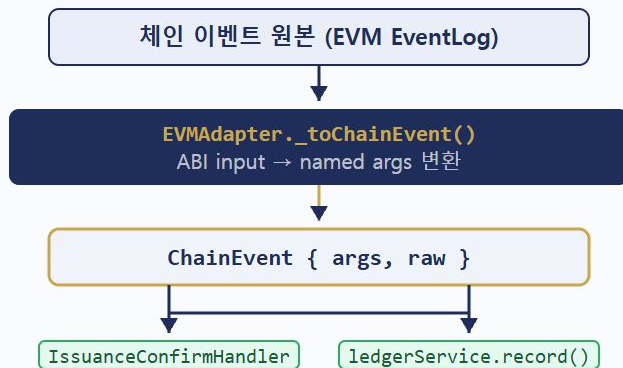
### ChainEvent — IBlockchainAdapter.ts:32

```
interface ChainEvent {  
  eventName: string;  
  contractAddr: string;  
  txHash: string;  
  blockNumber: number;  
  logIndex: number;  
  args: Record<string, unknown>; // 체인 무관 공통 필드  
  raw: unknown; // 체인별 원본 데이터 보존  
}
```

#### raw 사용 원칙

- ✅ 어댑터 내부에서 raw 파싱 → args 변환
- ❌ 어댑터 외부(IssuerService)에서 raw 직접 파싱

### raw 파싱 흐름 (EVMAdapter.ts:217)



핸들러에 ethers.js import 없음 → XRPL 교체 시 핸들러 변경 없음

# EVM vs XRPL vs Circle ARC 패러다임 비교

## BLOCKCHAIN PARADIGM COMPARISON

| 개념       | EVM (Ethereum)                                   | XRPL                        | Circle ARC         |
|----------|--------------------------------------------------|-----------------------------|--------------------|
| 스마트컨트랙트  | Solidity (Turing-complete)                       | 없음 (Hooks 실험적)              | 프로그래머블 월렛          |
| NFT 표준   | ERC-1155 (tokenId + amount)                      | XLS-20 (Offer 기반)           | ERC-1155 래핑        |
| 가스 체계    | $\text{Gas} \times \text{GasPrice} = \text{ETH}$ | Reserve(XRP 잠금) + fixed Fee | 가스 추상화 (Circle 처리) |
| 이벤트      | ABI-encoded Log (topics)                         | Transaction Metadata        | WebHook / SDK 이벤트  |
| Finality | PoS Checkpoint (~12분)                            | 3~5초 (BFT 합의)               | 체인 의존              |

**EVM ↔ XRPL 1:1 매핑 불가** — XRPL NFT 전송은 CreateOffer + AcceptOffer 두 단계. XRPLAdapter가 내부적으로 묶어서 하나의 `mintNFT()` 로 표현. IssuerService는 이 차이를 모른다.

# 멀티체인 전환 전략

## MULTICHAIN TRANSITION STRATEGY — ROADMAP

1

### Phase 1 (현재)

EVMAdapter  
Ethereum mainnet  
ERC-1155 NFT

2

### Phase 2 (국내 규제)

XRPLAdapter 병행  
국내 규제 대응  
코드 무변경

3

### Phase 3 (글로벌)

CircleAdapter  
글로벌 스테이블코인  
USDC/CCTP

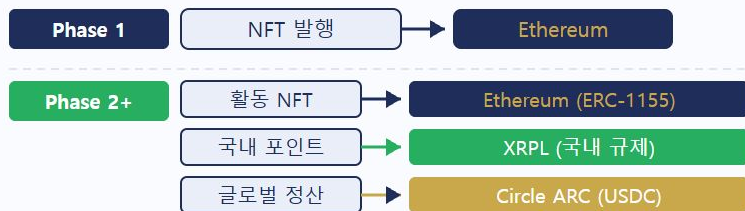
**핵심 원칙** — 비즈니스 로직·원장·감사 로그는 체인 무관하게 유지. 어댑터 구현체만 DI로 선택·주입한다.

**추상화의 조건** — 체인 교체가 실제 로드맵에 있을 때 장점이 비용을 초과한다. 근거 없으면 오버엔지니어링. 이 프로젝트는 Phase 1→2→3 전환이 실제 계획이므로 정당하다.

# 멀티체인 동시 운영 1/4

## MULTI-CHAIN SIMULTANEOUS OPERATION — REGISTRY PATTERN

### Phase 1 vs Phase 2+ 차이



세 어댑터가 같은 서비스 안에서 동시 동작

교체가 아닌 추가 — 기존 어댑터를 끄지 않고 새 어댑터를 레지스트리에 등록한다.

### ChainAdapterRegistry — 레지스트리 패턴

```
class ChainAdapterRegistry {  
    private readonly adapters =  
        new Map<string, IBlockchainAdapter>();  
  
    register(adapter: IBlockchainAdapter): void {  
        this.adapters.set(adapter.chainId, adapter);  
    }  
  
    get(chainId: string): IBlockchainAdapter {  
        const a = this.adapters.get(chainId);  
        if (!a) throw new UnsupportedChainError(chainId);  
        return a;  
    }  
  
    getAll(): IBlockchainAdapter[] {  
        return [...this.adapters.values()];  
    }  
}
```

# 멀티체인 동시 운영 2/4

## MULTI-CHAIN OPERATION — MULTI-ADAPTER ASSEMBLY

### 멀티 어댑터 조립 — index.ts (Phase 2+)

```
// 어댑터를 레지스트리에 등록 - 교체가 아닌 추가
const registry = new ChainAdapterRegistry();

registry.register(new EVMAAdapter({
  rpcUrl: process.env.EVM_RPC_URL!, chainId: 'sepolia',
}));
registry.register(new XRPLAdapter({ wsUrl: process.env.XRPL_WS_URL! }));
registry.register(new CircleAdapter({ apiKey: process.env.CIRCLE_API_KEY! }));

// TokenIssuerFactory에 registry 주입
const factory = new TokenIssuerFactory({ registry, vaspAdapter, coreBanking, pool });
```

비즈니스 코드 변경 없음 — TokenIssuerFactory · IssuerService는 registry를 통해 필요한 어댑터를 꺼낼 뿐. 어댑터가 몇 개든 관심 없다.

Phase 1과의 차이 — Phase 1: 단일 chainAdapter 주입. Phase 2+: registry 주입 → 다중 어댑터 동시 운영.



# 멀티체인 동시 운영 3/4

## MULTI-CHAIN OPERATION — POLICY-BASED ROUTING

### IssuerService 라우팅 — 정책 기반

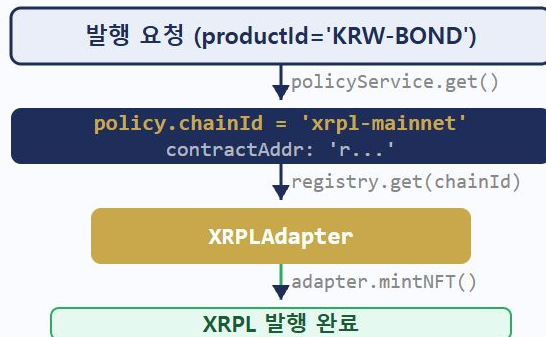
#### 정책 DB 기반 어댑터 선택

```
async issue(req: IssuanceRequest) {
  const policy = await
    this.policyService.get(req.productId);

  // policy.chainId = 'sepolia' | 'xrpl-mainnet' | ...
  const adapter =
    this.registry.get(policy.chainId);

  return adapter.mintNFT({
    contractAddr: policy.contractAddr,
    to: req.walletAddr, tokenId: req.tokenId,
    amount: 1n, requestId: req.requestId,
  });
}
```

#### 라우팅 흐름



비즈니스 코드 변경 없음 — 정책 DB에서 chain\_id가 바뀌면 다른 체인으로 자동 라우팅.



# 멀티체인 동시 운영 4/4

## MULTI-CHAIN OPERATION — INDEPENDENT LISTENERS PER CHAIN

### ChainEventListener — 체인 수만큼 독립 실행

```
const unsubscribes = await Promise.all(
  registry.getAll().map(adapter =>
    new ChainEventListener(adapter, handlers).start()
  )
); // EVM / XRPL / Circle 각각 독립적으로 subscribeEvents 실행

process.on('SIGTERM', () =>
  Promise.all(unsubscribes.map(unsub => unsub()))
);
```

#### 각 리스너의 독립성

- EVM 리스너 → Ethereum WebSocket 구독
- XRPL 리스너 → XRPL WebSocket 구독
- 하나가 실패해도 나머지 정상 동작
- GracefulShutdown 시 모두 동시 해제

#### 공통 핸들러 파이프라인

- 어떤 체인에서 오든 `ChainEvent` 동일 구조
- 같은 `handlers` 배열을 모든 리스너에 전달
- 핸들러 코드에 체인 분기 없음

**추상화 완성** — 이벤트 수신 경로도 체인 무관. 핸들러는 `ChainEvent` 만 본다.

# S14 핵심 요약

## KEY TAKEAWAYS

| 개념                            | 핵심                                                                |
|-------------------------------|-------------------------------------------------------------------|
| EVM 고착화 리스크                   | 체인 전용 개념이 비즈니스 로직에 스며들면 체인 교체 시 전체 수정 필요                          |
| Strategy Pattern              | 비즈니스 코드는 인터페이스만 의존 — 어댑터 교체는 DI 한 줄                               |
| 두 Factory의 역할                 | ChainAdapterFactory = 어댑터 생성, TokenIssuerFactory = 서비스 조립 — 책임 분리 |
| ChainEvent args / raw         | 어댑터 내부에서 raw 파싱 → named args 변환. 외부에서 raw 직접 파싱 금지                |
| subscribeEvents + queryEvents | 실시간 수신(Push) + 재시작 복구(Pull) 이중 채널 — 둘 다 있어야 유실 없음                 |
| 추상화의 조건                       | 체인 교체가 실제 로드맵에 있을 때 장점이 비용을 초과. 근거 없으면 오버엔지니어링                    |
| 멀티체인 동시 운영                    | 교체가 아닌 추가 — 레지스트리로 어댑터 관리, 비즈니스 코드 변경 없음 (Phase 2+)               |

**S15 예고** — 인터페이스를 배웠으니 이제 실제 EVMAdapter 구현을 분석한다. ethers.js v6 기반으로 sendTransaction → TransactionReceipt 변환, subscribeEvents unsubscribe 반환 패턴, XRPLAdapter stub 교체 시뮬레이션.

# S14 실습 — 섹션 구성

EXERCISE OVERVIEW — S14\_multichain\_adapter.ts

실행 방법 — `npm run exercise:s14` (전체) · `npm run exercise:s14:0` ~ `:s14:7` (개별)

| 섹션  | 이름                   | 확인 포인트                                      |
|-----|----------------------|---------------------------------------------|
| [0] | Sepolia 연결 확인        | EVMAdapter isConnected · getBlockNumber 실행작 |
| [1] | issuanceHealth       | 4개 어댑터에 동일 함수 적용 — 인터페이스만 사용                |
| [2] | formatGas            | gasUsed optional 처리 — EVM: 숫자 / Circle: N/A |
| [3] | withSubscription     | subscribeEvents 반환값이 함수인지 확인                |
| [4] | getFeeModel          | chainType 분기 — 4가지 수수료 모델                   |
| [5] | IssuerService        | 어댑터 교체해도 IssuerService 코드 변경 없음             |
| [6] | ChainAdapterFactory  | create() 분기 · UnsupportedChainError         |
| [7] | ChainAdapterRegistry | 멀티체인 동시 운영 · 정책 기반 라우팅                      |

[0]

# Sepolia 연결 확인

## EXERCISE 0 — EVMADAPTER CONNECTION CHECK

### 실습 코드

```
const adapter = new EVMAdapter({
  rpcUrl: evmConfig.rpcUrl,
  chainId: evmConfig.chainId,
  ...(evmConfig.signerKey && {
    privateKey: evmConfig.signerKey
  }),
});

const connected = await adapter.isConnected();
// connected가 true면 블록 번호 조회
if (connected) {
  const block = await adapter.getBlockNumber();
}
```

### 확인 포인트

- `isConnected()` → Sepolia RPC 실제 연결 여부
- 연결 성공 시 최신 블록 번호 출력
- `signerKey` 없어도 read-only로 동작

### 예상 출력

`isConnected: true`  
`blockNumber: 8xxxxxxx`

**RPC 미설정 시** — `isConnected: false`. 정상 케이스. 이후 섹션은 스텝으로 진행 가능.

[1]

# issuanceHealth — 4개 어댑터

## EXERCISE 1 — INTERFACE-ONLY FUNCTION

### issuanceHealth 함수

```
async function issuanceHealth(
  adapter: IBlockchainAdapter // ← 인터페이스만
): Promise<{ chain: string; type: string; connected: boolean }> {
  const connected =
    await adapter.isConnected().catch(() => false);
  return {
    chain: adapter.chainId,
    type: adapter.chainType,
    connected,
  };
}
```

```
// 4개 어댑터에 동일 함수 적용
const adapters: IBlockchainAdapter[] = [
  new EVMAAdapter(...),
  new XRPLAdapter(...),
  new CircleAdapter(...),
  new UTXOAdapter(...),
];
```

### 확인 포인트

- 함수 파라미터가 `IBlockchainAdapter` — 구현체 무관
- `.catch(() => false)` — 스텝 어댑터 연결 실패 안전 처리
- 4개 어댑터 모두 같은 함수로 처리됨

### 예상 출력

[EVM] chain=sepolia connected=true/false

[XRPL] chain=xrpl-testnet connected=false

[BFT] chain=circle-arc-mainnet connected=false

[UTXO] chain=bitcoin-mainnet connected=false

스텝 어댑터(Circle-UTXO)는 항상 false — 실환경에서는 정상 연결.

[2]

# formatGas — gasUsed optional 처리

## EXERCISE 2 — OPTIONAL FIELD HANDLING

### 실습 코드

```
function formatGas(
  receipt: TransactionReceipt
): string {
  return receipt.gasUsed !== undefined
    ? receipt.gasUsed.toString()
    : 'N/A';
}
```

```
// EVM receipt - gasUsed 있음
const evmReceipt: TransactionReceipt = {
  txHash: '0xee...', blockNumber: 1,
  status: 'success', gasUsed: 47704n, ...
};

// Circle receipt - gasUsed 없음
const circleReceipt: TransactionReceipt = {
  txHash: '0xc1c...', blockNumber: 0,
  status: 'success', ... // gasUsed 필드 없음
};
```

### 확인 포인트

- gasUsed 는 TransactionReceipt 에서 optional ( bigint? )
- EVM: 실제 가스 사용량 숫자
- Circle-XRPL: 가스 개념 없음 → N/A
- 호출하는 쪽이 optional 필드를 안전하게 처리하는 패턴

### 예상 출력

EVM gasUsed: 47704

Circle gasUsed: N/A

**잘못된 접근** — receipt.gasUsed.toString() 직접 호출 시  
Circle receipt에서 TypeError 발생.



[3]

# withSubscription — unsubscribe 함수 반환

## EXERCISE 3 — SUBScriBEEVENTS RETURN PATTERN

### 실습 코드

```
async function withSubscription(
  adapter: IBlockchainAdapter
): Promise<boolean> {
  const unsubscribe = await
    adapter.subscribeEvents(
      '', [], [], 0, async () => {}
    );

  // unsubscribe가 함수인지 확인
  const isFunction =
    typeof unsubscribe === 'function';
  unsubscribe(); // 즉시 해제
  return isFunction;
}
```

### 확인 포인트

- subscribeEvents() 의 반환 타입: `Promise<() => void>`
- 반환받은 값을 호출하면 구독 해제
- GracefulShutdown 패턴 — `process.on('SIGTERM', unsub)`

### 예상 출력

unsubscribe 함수 반환: true

왜 함수를 반환하는가 — EventEmitter 방식보다 간결. 리스너 참조를 직접 관리할 필요 없음. 해제 타이밍을 호출하는 쪽이 제어.

## [4] getFeeModel — chainType 분기

### EXERCISE 4 — CHAINTYPE BASED DISPATCH

#### 실습 코드

```
function getFeeModel(  
  adapter: IBlockchainAdapter  
) : string {  
  switch (adapter.chainType) {  
    case 'EVM':  
      return '가스(gas) 기반 — EIP-1559';  
    case 'XRPL':  
      return '고정 수수료 — XRP drops';  
    case 'BFT':  
      return '수수료 없음 — Circle 자체 부담';  
    case 'UTXO':  
      return 'UTXO 차액 — 채굴자 수수료';  
  }  
}
```

#### 확인 포인트

- chainType 은 IBlockchainAdapter 의 readonly 필드
- 각 어댑터 구현체가 고정값으로 선언
- TypeScript exhaustive switch — 모든 케이스 처리 강제

#### 예상 출력

[EVM] 가스(gas) 기반 — EIP-1559

[XRPL] 고정 수수료 — XRP drops

[BFT] 수수료 없음 — Circle 자체 부담

[UTXO] UTXO 차액 — 채굴자 수수료

**연결** — 슬라이드 24 패러다임 비교표의 가스 체계 항목이 코드로 표현된 것.

[5]

# IssuerService — 어댑터 교체 시 코드 변경 없음

## EXERCISE 5 — STRATEGY PATTERN VERIFIED

### 실습 코드

```
class IssuerService {
  constructor(private adapter: IBlockchainAdapter) {}

  async issue(to: string, tokenId: bigint) {
    return this.adapter.mintNFT({
      contractAddr: CONTRACT, to, tokenId,
      amount: 1n, requestId: `issue-${Date.now()}`,
    });
  }
}

// EVM으로 발행
const evmService = new IssuerService(
  new EVMAAdapter({...})
);

// Circle로 발행 - IssuerService 코드 동일
const circleService = new IssuerService(
  new CircleAdapter({...})
);
```

### 확인 포인트

- `IssuerService` 코드 한 줄도 바뀌지 않음
- 생성자에 다른 어댑터를 넣으면 다른 체인으로 발행
- Sepolia 연결 성공 시 실제 TX 전송, 아니면 스텝 receipt 반환

### 예상 출력

EVM : { txHash: '0xeee...', status: 'success', ... }

Circle : { txHash: '0xc1c...', status: 'success', ... }

**Strategy Pattern 완성** — 이것이 Strategy Pattern이 실제로 동작하는 모습이다.

[6]

# ChainAdapterFactory

## EXERCISE 6 — FACTORY PATTERN + UNSUPPORTED CHAIN ERROR

### 실습 코드

```
// EVM 어댑터 생성
const evm = ChainAdapterFactory.create({
  chainType: 'EVM',
  rpcUrl: evmConfig.rpcUrl,
  chainId: evmConfig.chainId,
});

// XRPL 어댑터 생성
const xrpl = ChainAdapterFactory.create({
  chainType: 'XRPL',
  rpcUrl: 'wss://s.altnet.ripple.net:51233',
  chainId: 'xrpl-testnet',
});

// 미지원 체인 → UnsupportedChainError
try {
  ChainAdapterFactory.create({
    chainType: 'UNKNOWN' as any, rpcUrl: '', chainId: ''
  });
} catch (err) { console.log(err.constructor.name, err.message); }
```

### 확인 포인트

- `ChainAdapterFactory.create()` — `chainType`으로 분기
- 반환 타입은 항상 `IBlockchainAdapter`
- 호출하는 쪽은 `EVMAdapter` 이름을 모른다
- 미지원 `chainType` → `UnsupportedChainError`

### 예상 출력

EVM : EVM sepolia

XRPL : XRPL xrpl-testnet

UNKNOWN → `UnsupportedChainError`: ...

**연결** — 슬라이드 18의 `default: never` 패턴이 런타임에서 이 예러로 나타난다.

# ChainAdapterRegistry — 멀티체인 동시 운영

## EXERCISE 7 — REGISTRY PATTERN IN ACTION

### 실습 코드 — 3가지 시나리오

```
// 1. getAll() - 모든 어댑터 동시 조회
const results = await Promise.all(
  registry.getAll().map(a => issuanceHealth(a))
);

// 2. get(chainId) - 정책 기반 라우팅
const evmAdapter = registry.get(evmConfig.chainId);
// → 'sepolia' chainId로 EVMAdapter 반환

// 3. 미등록 chainId → UnsupportedChainError
try {
  registry.get('cosmos-mainnet');
} catch (err) { /* UnsupportedChainError */ }

// 4. ChainEventListener × N - 독립 구독
const unsubs = await Promise.all(
  registry.getAll().map(a => withSubscription(a))
);
```

### 확인 포인트

- register() → chainId를 키로 Map에 저장
- getAll() → 등록된 모든 어댑터 동시 처리 가능
- get(chainId) → 정책 DB의 chain\_id로 어댑터 선택
- 미등록 → UnsupportedChainError
- subscribeEvents × N → 체인마다 독립 구독

### 예상 출력

[EVM] chainId=sepolia connected=true/false

[XRPL] chainId=xrpl-testnet connected=false

'cosmos-mainnet' → UnsupportedChainError

→ 4/4개 체인 구독 완료